Benyamin Gidanian

Los Angeles, CA 90034 · (310) 613-3655 · bgidanian@gmail.com
GitHub · LinkedIn

Summary

Applied AI/ML engineer who builds and ships systems end-to-end — from fine-tuning and evaluating models to deploying them as live, production services. Comfortable across the full stack: PyTorch model development, Python backend services (FastAPI, Django), Docker containerization, CI/CD, and cloud deployment (AWS ECS Fargate). Background spans NLP (transformer fine-tuning, token classification), computer vision (transfer learning, class imbalance), and full-stack web development. Moves fast, experiments with new tools and approaches, and focuses on shipping things that work.

Technical Skills

Core ML/AI: PyTorch, TensorFlow, Keras, CUDA, Torchvision, BERT, Attention Mechanisms, Hugging Face Transformers, timm, ConvNeXt
Algorithms: Supervised & Unsupervised Learning, CNNs, Neural Networks, Decision Trees, XGBoost, Recommender Systems, Reinforcement Learning (DQN)
ML Engineering: FastAPI, Docker, RESTful APIs, CI/CD (GitHub Actions), Transfer Learning, Hyperparameter Tuning, Model Evaluation (ROC/AUC, Precision, Recall, F1)
Cloud/Infra: AWS ECS Fargate, ECR, IAM, SSM Parameter Store, Cloudflare (DNS/TLS)
Full-Stack / Web: Django, React, REST API design, SQL (SQLite)
Data: NumPy, Pandas, Feature Engineering, Data Preprocessing
Languages: Python, SQL, JavaScript, C
Tools: Git, Linux, Bash, Pytest, Ruff, Pyright, uv, Jupyter Notebooks, HuggingFace Hub

---

Projects

PII Redaction

PyTorch · DeBERTa-v3 · HuggingFace Trainer/Hub · FastAPI · Docker · AWS ECS Fargate · GitHub Actions CI/CD
GitHub · Live Demo

Models: base · small · xsmall

- Fine-tuned three DeBERTa-v3 models on `ai4privacy/pii-masking-300k` for PII named entity recognition across 27 entity types (55 BIO labels: names, emails, SSNs, phone numbers, etc.), achieving macro F1 of **0.9557 (base)**, **0.9517 (small)**, and **0.9424 (xsmall)** on the held-out test set.
- Built a custom `WeightedTokenClassificationTrainer` with class-balanced cross-entropy loss to handle severe O-token imbalance; trained in two phases (frozen backbone → full fine-tune) using bf16 mixed precision, with W&B logging including per-entity F1 and interactive HTML prediction tables.
- Implemented a `PIIRedactor` class with overlapping token chunking (paragraph/sentence-aware boundary alignment), overlap resolution by max confidence score, and structured Pydantic `Entity` / `RedactionResponse` schemas; exposed via a CLI supporting `--text`, `--file`, threshold tuning, stride/max-length overrides, and JSON output.
- Built a FastAPI service with API-key-authenticated `POST /redact`, `GET /health`, `GET /model-info`, and an interactive dark-mode `GET /demo` with glassmorphism layout, drag-and-drop file upload,

and confidence badges; `POST /demo/redact` is unauthenticated but rate-limited (10/day per IP via slowapi). 14 unit tests mock the redactor across auth, rate limiting, validation, and edge cases.
- Published all three model checkpoints to HuggingFace Hub with full model cards documenting per-model F1, GPU latency benchmarks, distribution-shift behavior, and per-entity tradeoffs.
- Deployed to **AWS ECS Fargate** behind a custom domain with Cloudflare TLS termination and dynamic DNS updates on every deploy; secrets managed via SSM Parameter Store. Built a full GitHub Actions CI/CD pipeline (Ruff, Pyright, Pytest, Docker build, ECR push, ECS service update) that ships every push to `main` straight to production.

Flower Classification Inference Service

PyTorch · EfficientNet · FastAPI · Docker · HuggingFace Hub · GitHub Actions
GitHub · Model

- Achieved **>93% test accuracy** on the 102-class Oxford Flowers dataset via two-stage transfer learning of EfficientNet-B0 (classifier-only warm-up, then partial backbone unfreeze with differential learning rates).
- Served the model via a FastAPI service (`/predict` , `/health`) with file-type, size, and image-dimension validation; integrated automatic weight download from HuggingFace Hub on cold start.
- Containerized with Docker with a full CI pipeline (GitHub Actions): Ruff, Pyright, Pytest (model-mocked for GPU-free CI), and Docker build verification on every push.

Network — Full-Stack Social Platform

Django · React · SQLite · JavaScript GitHub

- Built a Twitter-style social platform with a Django backend (custom User, Post, Like, and Follow models, including many-to-many relations via through-models) and a React-powered frontend.
- Designed RESTful endpoints for authentication, posting, following, and liking, with server-side validation and Fetch API calls for asynchronous, no-reload post editing, liking, and pagination.

Gravitational Lens Finding — ML4SCI DeepLense (GSoC 2026 Evaluation)

PyTorch · ConvNeXt-Tiny · EfficientNet-B0 · timm GitHub

- Achieved **0.9887 AUROC** on binary gravitational lens classification (16.6:1 class imbalance) using ConvNeXt-Tiny, and **0.98 macro AUROC** on 3-class substructure classification via two-phase fine-tuning of EfficientNet-B0.
- Addressed severe class imbalance with combined `WeightedRandomSampler` oversampling and calibrated `BCEWithLogitsLoss` pos-weighting.

Additional Projects

- **ML Fundamentals Library** *(Python, NumPy, Pytest)* — Built a from-scratch ML library (Linear/Logistic Regression, K-Means, Decision Trees, multi-layer NN with manual backprop) with 100% test coverage and a custom AST-based doc generator. GitHub
- **DQN for Autonomous Lunar Landing** *(TensorFlow, Keras, Gymnasium)* — Implemented a DQN agent with experience replay and soft-update target networks, solving LunarLander-v3 with a rolling average score above 200. GitHub
- **Crossword Generator** *(Python, CSP/AC-3)* — Built a constraint satisfaction solver using AC-3 arc consistency plus MRV/LCV heuristics, reducing backtracking search space by over 80%. GitHub

---

Professional Experience

One-on-One Talmud Tutor

Mesivta Bircas Yitzchok · Sept 2023 – March 2025

- Delivered individualized instruction over a 1.5-year engagement, developing structured curricula to accelerate comprehension and long-term retention.

Education & Certifications

Machine Learning Specialization — Stanford University & DeepLearning.AI (Coursera)
Supervised Learning, Neural Networks, Decision Trees, Unsupervised Learning, Recommender Systems

Harvard CS50 Suite — Harvard University (EdX)
CS50x (C, Python, SQL, Algorithms) · CS50p (OOP, Testing) · CS50ai (Search, Logic, Optimization) · CS50w (Django, React, CI/CD)